

Clase 5: La Web Viva - APIs y Fetch

JavaScript ⚡

En esta clase aprenderemos a conectar el Frontend y el Backend de forma asíncrona. Esto significa que nuestra página podrá pedir y enviar datos al servidor en segundo plano, sin necesidad de refrescar la ventana. Importante hay que crear una nueva base de datos

1. ¿Qué es una API REST?

Imagina que el Backend es la **cocina** de un restaurante y el Frontend es el **comensal**. El cliente no entra a la cocina a buscar su comida; hay un **mesero** que lleva el pedido y trae el plato.

- **API (Application Programming Interface):** Es ese mesero. Es una dirección URL que no devuelve una página visual, sino datos puros (generalmente en formato **JSON**).

2. El poder de fetch() en JavaScript

fetch() es la función nativa de JavaScript que nos permite hacer peticiones HTTP desde el navegador hacia nuestro servidor de Python sin recargar la página. Para ello debemos llamar la extensión jsonify de la siguiente forma:

```
from flask import Flask, render_template, request, redirect, url_for, jsonify
```

El flujo es simple:

1. JavaScript hace un fetch a una ruta de Python.
2. Python procesa la información y responde con un JSON.
3. JavaScript recibe el JSON y actualiza el HTML automáticamente.

3. Guía de Montaje Paso a Paso

Paso 1: Crear la "Base de Datos" en Memoria

Para esta práctica, usaremos una lista de diccionarios en Python que representará nuestros "servicios" o "tareas". Cada vez que el usuario escriba, consultaremos esta lista. En este caso modificaremos el modelo de datos:

En la clase Producto hay que añadir otro elemento a parte de id, nombre y precio, crearemos el elemento categoria:

```
categoria = db.Column(db.String(50), nullable=True) # Categoría opcional
```

Hay que modificar el llamado de:

```
def __repr__(self):  
    # return f'<Producto {self.nombre}>'
```

por este otro:

```
def to_dict(self):  
    """Convierte el objeto a un diccionario para poder enviarlo como JSON"""  
    return {  
        "id": self.id,  
        "nombre": self.nombre,  
        "precio": self.precio,  
        "categoria": self.categoria  
    }
```

Paso 2: Crear el Endpoints en Python

Necesitaremos una ruta especial que en lugar de render_template, use jsonify.

- **Ejemplo:** /api/buscar?q=limpieza devolverá solo los servicios que coincidan con esa búsqueda.

```
@app.route('/api/buscar')
def buscar_productos():
    """
    Esta es la ruta que usará JavaScript.
    Busca en la base de datos real sin recargar la página.
    """
    query_text = request.args.get('q', "").lower()

    if not query_text:
        return jsonify([])

    # Buscamos en la base de datos (SQLAlchemy)
    # Filtramos donde el nombre contenga el texto buscado
    resultados = Producto.query.filter(Producto.nombre.ilike(f'%{query_text}%')).all()

    # Convertimos los resultados a una lista de diccionarios
    return jsonify([p.to_dict() for p in resultados])
```

Paso 3: El "Escuchador" en JavaScript (Frontend)

En el archivo HTML, agregaremos un input de búsqueda. Usaremos el evento keyup o input.

- Cada vez que el usuario presione una tecla, JavaScript capturará el texto y lo enviará al servidor mediante fetch.

Paso 4: Actualización Dinámica del DOM

Cuando recibamos la respuesta del servidor, borraremos la lista actual en el HTML y dibujaremos la nueva lista de resultados usando un bucle en JavaScript.

```
<div class="max-w-3xl mx-auto">

  <header class="mb-10 text-center">

    <h1 class="text-4xl font-black text-slate-900 tracking-tight">Catálogo Interactivo</h1>

    <p class="text-slate-500">Busca productos en tiempo real usando Fetch API</p>

  </header>

  <!-- BUSCADOR -->

  <div class="relative mb-12">

    <input type="text" id="inputBusqueda" placeholder="Empieza a escribir el nombre de un producto..."

      class="w-full p-5 rounded-2xl shadow-lg border-none ring-1 ring-slate-200 focus:ring-4 focus:ring-indigo-500 outline-none text-lg transition-all">

    <div id="loader" class="absolute right-5 top-5 hidden">

      <div class="animate-spin rounded-full h-7 w-7 border-b-2 border-indigo-600"></div>

    </div>

  </div>

  <!-- RESULTADOS -->

  <div id="contenedor-resultados" class="grid gap-6">

    <div class="text-center py-20 bg-white rounded-3xl border-2 border-dashed border-slate-200 text-slate-400">

      <p>Escribe en el buscador para ver los resultados de la base de datos.</p>

    </div>

  </div>
```

</div>

</div>

<script>

const input = document.getElementById('inputBusqueda');

const contenedor = document.getElementById('contenedor-resultados');

const loader = document.getElementById('loader');

async function realizarBusqueda(texto) {

if (texto.length < 2) {

contenedor.innerHTML = '<div class="text-center py-20 bg-white rounded-3xl border-2 border-dashed border-slate-200 text-slate-400"><p>Escribe al menos 2 letras...</p></div>';

return;

}

loader.classList.remove('hidden');

try {

// Llamamos a nuestra nueva ruta de Python

const response = await fetch(`/api/buscar?q=\${texto}`);

const productos = await response.json();

contenedor.innerHTML = "";

if (productos.length === 0) {

contenedor.innerHTML = '<p class="text-center text-slate-500">No se encontraron coincidencias.</p>';

} else {

productos.forEach(p => {

const card = `

<div class="bg-white p-6 rounded-2xl shadow-sm border border-slate-100 flex

```

justify-between items-center hover:scale-[1.02] transition-transform">

    <div>

        <h3 class="text-xl font-bold text-slate-800">${p.nombre}</h3>

        <p class="text-slate-400 text-sm uppercase font-bold
tracking-tighter">${p.categoria}</p>

    </div>

    <div class="text-right">

        <p class="text-2xl font-black text-indigo-600">${p.precio.toFixed(2)}</p>

        <span class="text-xs text-slate-300 font-mono">ID: #${p.id}</span>

    </div>

</div>

`;

contenedor.innerHTML += card;

});

}

} catch (error) {

    console.error("Error en la petición:", error);

} finally {

    loader.classList.add('hidden');

}

}

// Escuchar el teclado

input.addEventListener('input', (e) => {

    realizarBusqueda(e.target.value);

});

</script>

```

Conceptos Clave para el Éxito

- **Event Loop:** Entender que la petición fetch es una "promesa"; toma tiempo y debemos esperar a que el servidor responda.
- **JSON (JavaScript Object Notation):** Es el lenguaje universal que entienden tanto Python como JavaScript para intercambiar información.
- **Manipulación del DOM:** Usaremos `document.getElementById` e `innerHTML` para inyectar los resultados que vienen del servidor.

Actividad Sugerida: "Buscador Predictivo"

El reto para los estudiantes es crear un buscador que, al escribir "Plomería", filtre instantáneamente los servicios disponibles sin que el usuario tenga que presionar un botón de "Buscar".